

Análise Visual de Erros de Ponto Flutuante e das Units of Last Place em Experimentos Computacionais

Enzo Rocha Leite Diniz Ribas¹, D'Afonseca, L. A.², Rocha, L. M.³

Centro Federal de Educação Tecnológica de Minas Gerais, MG, Brasil

enzorochaleitedinizribas@gmail.com¹, luis.dafonseca@gmail.com², lucasmartinsrocha@gmail.com³

Resumo

Este trabalho investiga, por meio de experimentos computacionais, os efeitos da representação em ponto flutuante e das *Units in the Last Place* (ULP's). Foram analisadas funções matematicamente exatas cuja avaliação em aritmética de ponto flutuante produz erros de arredondamento, com ênfase nos efeitos associados à escala dos números representados e à resolução finita do sistema. Os experimentos foram implementados em Python e utilizados para gerar representações gráficas dos fenômenos observados. Os resultados evidenciam a influência da ULP na perda de significância e na alteração do comportamento numérico das funções avaliadas.

Palavras-Chave: Erros Numéricos; IEEE 754; Métodos Numéricos; Cancelamento Catastrófico; Ponto Flutuante.

Introdução

Frequentemente, computadores são utilizados para realizar cálculos numéricos repetitivos e de grande complexidade. Apesar disso, a representação computacional de números reais é limitada pela quantidade finita de bits disponível, fazendo com que operações aritméticas estejam sujeitas a erros de arredondamento. Essa limitação torna-se particularmente relevante à medida que a magnitude dos números aumenta, pois o espaçamento entre valores representáveis, ou *Unit in the Last Place* (ULP), também aumenta.

Neste trabalho, busca-se analisar experimentalmente a influência da representação em ponto flutuante e da ULP sobre o comportamento numérico de funções, investigando a relação entre a magnitude dos valores representados, os erros de arredondamento e a perda de significância. Para isso, são realizadas simulações computacionais e representações gráficas que permitem identificar e visualizar as regiões em que a aritmética de ponto flutuante produz resultados diferentes dos esperados pela aritmética exata.

Fundamentação Teórica

Um Sistema de Ponto Flutuante (SPF) é a maneira utilizada para aproximar números reais pelos computadores através de representações binárias compactas. Para a maioria dos computadores atuais, o padrão adotado é o definido pela IEEE-754 [4]. Definimos, então, a função $f(x)$ que leva um número real x a sua representação em ponto flutuante.

Devido à natureza discreta e finita dos SPF's, erros de representação são intrínsecos e propagam-se em operações aritméticas sucessivas, acumulando-se e podendo comprometer severamente a exatidão numérica das operações.

No padrão IEEE 754 um número de ponto flutuante é dividido em três partes. O primeiro bit, representa o **Sinal (S)** do número representado, indicando se o número é positivo ($S = 0$) ou negativo ($S = 1$); Os próximos N bits representam o **expoente (E)** com viés (bias), que é calculado como $\text{bias} = 2^{(N-1)} - 1$; E por fim, os últimos M bits Representam a **Mantissa (M)** que é parte fracionária significativa do número.

$$f(x) = (-1)^S \times (1.M) \times 2^{E-\text{bias}}, \quad (1)$$

Na mantissa, o termo $1.M$ indica que há um bit implícito "1" antes da mantissa nos números normalizados.

A *Unit of Last Place* (ULP) é a menor diferença entre dois números representáveis em ponto flutuante, ou seja, a distância entre um número e o próximo número representável [1, 2]. O conjunto de números exatamente representáveis se torna mais esparsos quanto mais distante o número for de zero, aumentando o erro relativo. Dessa forma, a ULP se torna uma medida relativa importante para avaliar a precisão dos cálculos realizados com ponto flutuante, podendo ser calculada como $\text{ULP}(x) = 2^{p-t+1}$ em que p é o expoente do número representado em ponto flutuante e t é o número de bits da mantissa.

Para analisar o comportamento das ULP's, foram realizados experimentos computacionais utilizando funções da forma $g(x) = x^{10} + \ell - x^{10}$, cujo valor exato é constante e igual a ℓ . A implementação foi realizada utilizando a biblioteca *numpy* e outras bibliotecas para recursos gráficos para visualizar o comportamento numérico da função.

Pela natureza binária do sistema de ponto flutuante, analisar diferentes potências de base 2 para ℓ revela uma boa visualização da grandeza da $\text{ULP}(x)$ conforme x cresce.

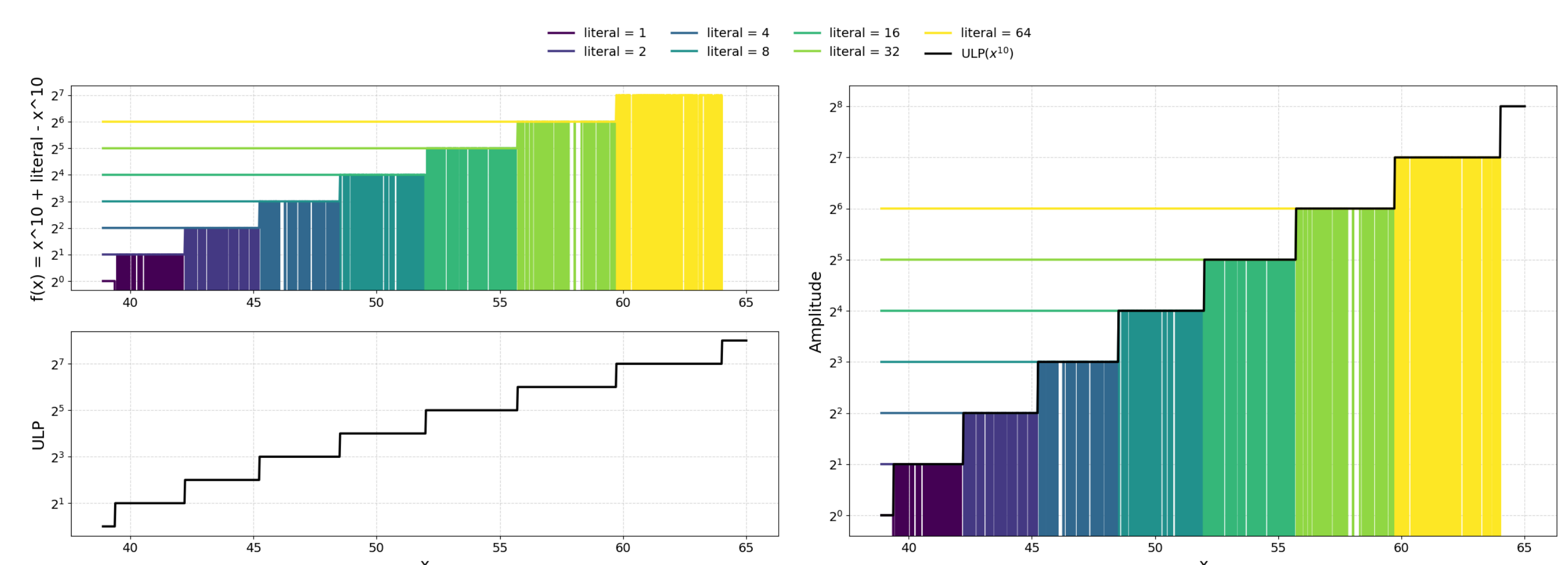


Figura 1: Resultado da avaliação da função $f(x) = x^{10} + l - x^{10}$ num SPF IEEE de 64 bits para $\ell = 2^n, n \in \mathbb{N}, n \leq 7$. Fonte: Autor.

Ao avaliar g para diferentes valores de ℓ , usando ponto flutuante, obtemos o gráfico da $\text{ULP}(x)$, ilustrado na Figura 1, em que o valor esperado $g(x) = \ell$ é exibido apenas até um certo x , e a partir de um intervalo, a função assume o valor zero. Isso ocorre, pois o valor x^{10} é muito maior que ℓ , o que resulta na perda de significância, produzindo $x^{10} + \ell = x^{10}$. Entretanto, a ocorrência no intervalo intermediário ao valor esperado e o zero é observada uma oscilação relacionada surpreendente, que tem amplitude igual a $\text{ULP}(x^{10}) = \ell$.

Resultados

Considere a função $f(x) = g(x)$ com $\ell = 1$. Ao avaliar f utilizando aritmética de ponto flutuante, obtemos o gráfico da Figura 2. O intervalo intermediário de oscilação é de aproximadamente [43, 49].

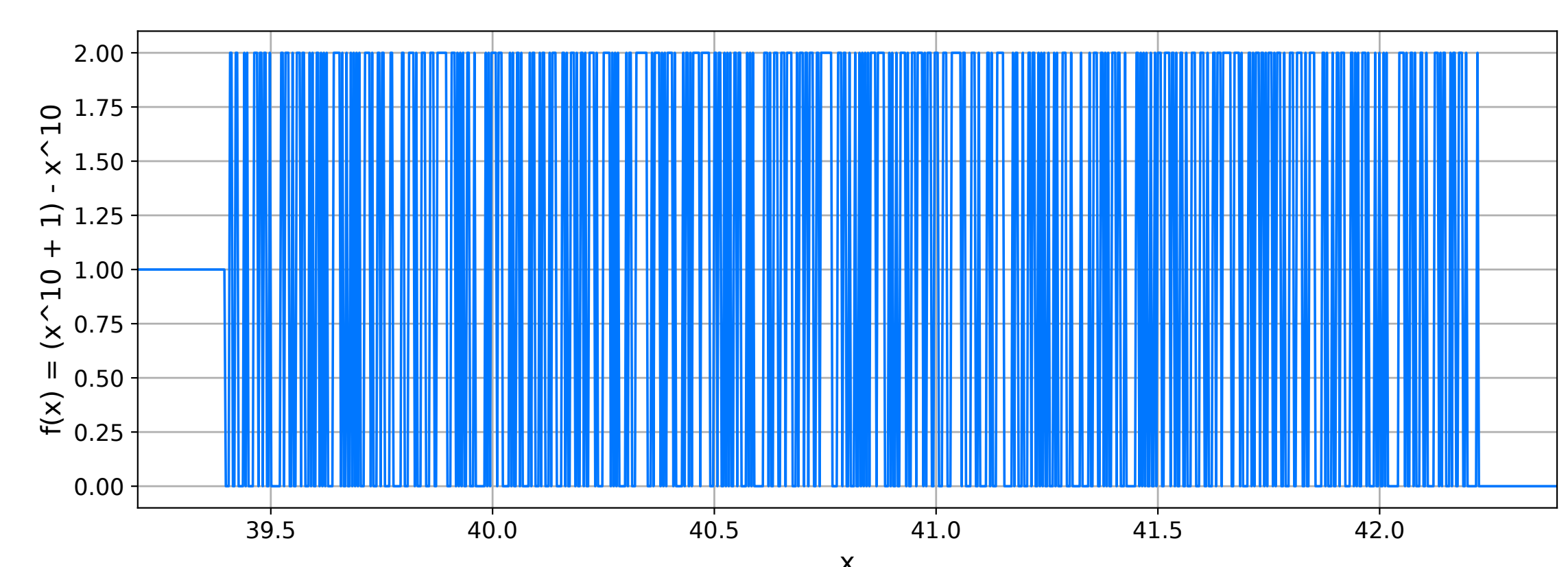


Figura 2: Resultado da avaliação da função $f(x) = x^{10} + 1 - x^{10}$ num SPF IEEE de 64 bits. Fonte: Autor.

Tomemos como exemplo o valor $x = 42,1$. Na matemática exata, a expressão avalia para:

$$x^{10} = 17.491.254.479.245.335,8346502201 \dots$$

Devido às limitações de resolução do sistema, x não possui representação exata. O cálculo computacional inicia-se com a aproximação $f(x) \approx 42,100000000000001$. Elevando este valor à décima potência, obtemos o valor flutuante A :

$$A = f((f(x))^{10}) = 17.491.254.479.245.342$$

A estrutura binária de A no padrão IEEE 754 (64 bits) é dada por:

S	E	M
0	100 0011 0100	1111 0001 0010 0001 1010 0000 0100 1111 0100 1010 1000 0000 1111

Ao calcularmos a adição $B = A + 1$, o resultado exato (17.491.254.479.245.343) cai exatamente no meio de dois números de exatamente representáveis. O padrão IEEE 754 resolve esta ambiguidade utilizando a regra *Round to Nearest, Ties to Even* (arredondar para o mais próximo; em caso de empate, para o par, onde o *Least Significant Bit* é nulo).

Analisando os dois candidatos à representação de B em binário, suprimindo o início para focar no final da mantissa temos:

Opção 1 (A) : [...]0100 1010 1000 0000 **1111**₂

Opção 2 ($A + 2$) : [...]0100 1010 1000 0001 **0000**₂

Como o bit menos significativo (LSB) da segunda opção é 0 (par), o sistema desempata arredondando para este valor. Logo, $f(B) = 17.491.254.479.245.\mathbf{344} = A + 2$.

Por fim, ao subtrair o termo x^{10} previamente calculado, temos a resolução final em máquina:

$$f(42, 1) = f(x^{10}) + f(1) - f(x^{10})$$

$$= 17.491.254.479.245.\mathbf{344} - 17.491.254.479.245.\mathbf{342}$$

$$= 2$$

Este salto de representação e arredondamento forçado fornecem a justificativa analítica exata para a oscilação verificada no intervalo intermediário.

Conclusões

Os experimentos computacionais conduzidos demonstraram com êxito como a resolução finita do padrão IEEE 754 afeta a exatidão de algoritmos. A metodologia de análise visual permitiu não apenas observar o esperado cancelamento catastrófico em valores de alta magnitude, mas, sobretudo, identificar e mapear as regiões intermediárias de oscilação, confirmando a hipótese inicial de instabilidade atrelada à escala.

A inspeção em baixo nível da estrutura binária provou que a anomalia visualizada não tem natureza estocástica; trata-se de uma consequência algorítmica e determinística

das regras de arredondamento, especificamente o critério *Round to Nearest, Ties to Even*. O crescimento da ULP atua diretamente na amplitude desses erros, transformando operações analiticamente triviais em potenciais pontos de falha silenciosa no código.

Do ponto de vista da Engenharia de Software e da Computação Científica, os resultados evidenciam que a modelagem matemática exata frequentemente falha no domínio da máquina. A confiabilidade de simulações físicas, métodos numéricos e integrações de baixo nível exige que o desenvolvedor reescreva expressões algébricas considerando a topologia discreta do Sistema de Ponto Flutuante.

Como desdobramentos lógicos para trabalhos futuros, propõem-se:

- **Exploração de novos formatos:** Avaliar o comportamento do cancelamento utilizando arquiteturas de dados emergentes, como `bfloat16` e *Posits* (Type III Unums).
- **Ferramental de instrumentação:** Desenvolver uma biblioteca estática (C/C++) para o monitoramento em tempo de execução da oscilação de ULPs em malhas de cálculo crítico, alertando sobre a perda de significância antes da consolidação do erro.

Referências

[1] William Kahan. A logarithm too clever by half. *Unpublished note*, 2004.

[2] Jean-Michel Muller. On the definition of ulp(x). *CNRS – Laboratoire LIP, projet Arenalire*, 2005.

[3] Márcia Aparecida Gomes Ruggiero and Vera Lúcia da Rocha Lopes. *Cálculo numérico: aspectos teóricos e computacionais*. Pearson/Makron, Sao Paulo, 1998.

[4] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*, 2019.